Multi-Task Offline Reinforcement Learning Project

Siyu Pan, Xiaohan Yu, Yuchen Hou

Abstract

We introduces Batch Constrained Conservative Q-Learning (BC-CQL), a novel approach tailored for multi-task offline reinforcement learning. BC-CQL enhances learning stability by minimizing extrapolation errors, which are prevalent in offline settings. We compare BC-CQL against established offline learning algorithms such as TD3-BC, BCQ, and CQL, demonstrating its superior performance in terms of convergence speed and stability. Additionally, we investigate a Conservative Data Sharing (CDS) method that optimizes data utilization across multiple tasks, further improving policy quality. Preliminary results indicate that while BC-CQL shows promising improvements over traditional methods, the limited training duration poses challenges for comprehensive evaluation, suggesting avenues for future research.

1 Introduction

1.1 Offline Reinforcement Learning

Reinforcement learning (RL) provides a mathematical formalism for learning-based control. By utilizing RL, we can automatically acquire near-optimal behavioral skills. However, the fact that RL algorithms provide a fundamentally online learning paradigm is also one of the biggest obstacles to their widespread adoption.

The offline reinforcement learning can utilize only previously collected offline data, without any additional online interaction, overcoming the costly, risky, and time-consuming data collection process of online learning. Recent advances in offline reinforcement learning make it possible to train policies for real-world scenarios, such as robotics and healthcare. However, the offline RL also faces several challenges.The distributional shift between the optimal policy and the effective behavior policy in offline RL will cause extrapolation error, leading to off-policy agents perform dramatically poor. Algorithms like conservative Q-learning, batch constrained Q-learning and TD3-BC were proposed to tackle this problem by regularizing the learned policy to be "close" to the behavior policy.

1.2 Multi-task Offline Reinforcement Learning

Many real-world scenarios for applying offline RL involve multi-task problems, where the goal is to solve multiple tasks using all available data. Existing datasets in fields like robotics also often contain heterogeneous data collected for different tasks. Previous multi-task RL algorithms focused on solving multiple tasks jointly in an efficient way. Although multi-task RL methods seem to provide a promising way to build general-purpose agents, current work mostly faced two challenges: the difficulties in model optimization and the exacerbated the distributional shift between the optimal policy and the effective behavior policy induced by data sharing.

1.3 Previous Work

CQL Conservative Q-Learning (CQL)**?** is a strategy in offline reinforcement learning aimed at addressing the over-estimation of Q-values. CQL ensures the expected value under the Q-function

Submitted to 38th Conference on Neural Information Processing Systems (NeurIPS 2024). Do not distribute.

lower-bounds its true value by introducing a conservative update mechanism. This method not only suppresses the overestimation inherent in offline settings but also stabilizes the learning process.

The primary formulation of CQL involves modifying the standard Bellman error objective by incorporating a Q-value regularizer. This regularization discourages deviations from the behavior policy distribution and can be formalized as follows:

$$\min_{Q} \max_{\mu} \Big\{ \alpha \left(\mathbb{E}_{s\sim D, a\sim\mu(a|s)}[Q(s,a)] - \mathbb{E}_{s\sim D, a\sim\pi(a|s)}[Q(s,a)] \right) + \frac{1}{2}\mathbb{E}_{(s,a,s')\sim D}\left[\left(Q(s,a) - \mathcal{B}^{\pi}Q^{k}(s,a) \right)^2 \right] + \mathcal{R}(\mu) \Big\}$$

$\mathcal{R}(\mu)$ represents a regularization term, often chosen as the KL divergence between the current policy $\mu(a|s)$ and a predefined prior distribution, which in some cases may be the uniform distribution over actions.

Further, the regularization component $\mathcal{R}(\mu)$ is crucial for controlling the policy's deviation from feasible actions as per the offline data, thereby helping to mitigate common pitfalls such as distributional shift and extrapolation errors.

CDS Conservative Data Sharing (CDS)**?** is an approach in RL designed to improve performance in multi-task settings by selectively sharing data across tasks. Unlike naive data sharing, which can degrade performance by introducing irrelevant or conflicting information, CDS carefully selects which data to share based on its relevance and potential benefit to each task. CDS relabels a transition into a given task only when it is expected to improve performance based on a conservative estimate of the Q-function. After data sharing, similarly to prior offline RL methods, CDS applies a standard conservative offline RL algorithm, such as CQL that learns a conservative value function or BRAC, a policy-constraint offline RL algorithm. This method aims to leverage the advantages of larger datasets while mitigating the risks of data heterogeneity, ultimately enhancing the robustness and effectiveness of RL models in offline settings.

BCQ BCQ**?** attempts to eliminate extrapolation error by constraining the agent's actions to the data distribution of the batch. Batch-constrained policies are trained to select actions with respect to three objectives: 1) Minimize the distance of selected actions to the data in the batch 2) Lead to states where familiar data can be observed. 3) Maximize the value function. BCQ approaches the notion of batch-constrained through a generative model, which generates plausible actions with high similarity to the batch, and then selects the highest valued action through a learned Q-network. VAE is used as a generative model in BCQ, The generative model G, alongside the value function Q, can be used as a policy by sampling n actions from G and selecting the highest valued action according to the value estimate Q.

2 Algorithm

2.1 Environment

Our experiments are conducted in a highly customized simulation environment based on the MuJoCo Walker2D framework. The simulated agent, a bipedal walker, is designed with seven main body parts to control. The primary objective for the agent within this environment is to achieve either walking or running, leveraging these articulated body segments. The state space of the environment is defined with 24 dimensions, encompassing various physical and kinematic properties.The action space is constrained to 6 dimensions, each corresponding to torques applied at the hinge joints. Both tasks, walking and running, share identical state spaces, action spaces, and transition probabilities, differing only in their respective reward functions designed to optimize task-specific performance metrics.

The dataset used for training comprises offline samples from two distinct tasks: walking and running. Each task is represented by two subsets of data quality: medium and medium-replay. These samples are extracted from the replay buffer, ensuring a diverse range of scenarios based on previous interactions of the agent with the environment. This offline dataset serves as the exclusive source, which, during the project, adapts solely based on this historical data without further online interaction.

Our goal is to empower the multi-task agent to effectively learn from this comprehensive dataset and demonstrate robust performance across both designated tasks during evaluation phases.

2.2 Data Sharing Strategy

In this work, we used three data sharing strategies to construct the training dataset, including no data sharing, direct data sharing and conservative data sharing. For no sharing, the dataset is set only the main task dataset. For direct data sharing, we directly mixed the dataset of the main task with the dataset of the other task. For conservative data sharing, we adapted the data sharing strategy of CDS, to ensure the distributional shift is strictly reduced when mixing the data.

2.3 Pipeline

Our pipeline is shown in the Figure 1.effectively filtering the data in shared task and

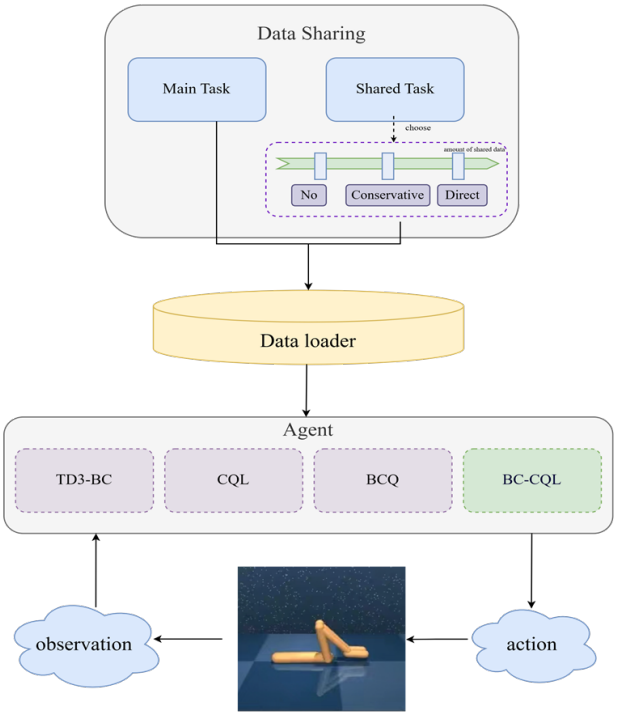


Figure 1: pipeline

We utilize different data sharing method to optimize the data loader component and employ several classic RL algorithms to leverage the offline data:

- **TD3-BC:** Combines TD3 and Behavioral Cloning to exploit the dataset while maintaining policy robustness.
- **Conservative Q-Learning (CQL):** Focuses on avoiding overestimation of action values outside the support of the data distribution.
- **Batch-Constrained deep Q-learning (BCQ):** Restricts actions to those similar to the behavior policy, reducing the likelihood of detrimental actions.

Then we propose a novel agent call **BC-CQL**. This algorithm is enlightened by the CQL and BCQ. We attached the Batch Constraint (BC) policy to CQL. Specifically, we first pretrained a VAE model on the walk-medium-replay dataset.The input of the VAE is the observation from walk-medium-replay dataset, and the output is the corresponding action. This VAE helps us to generate actions close to the real action since the begin of training, thus both reducing the distribution shift caused by data sharing

and preventing the agent being stuck because of unfamiliar observation. When updating the actor, the original CQL agent samples new actions based on the observation and the learned policy. However, the addition of BC makes the sampled actions become the output of the VAE. In order to make the model backpropagate correctly, we concentrated the actions generated by VAE with the observation, as the new input of the actor.

For all types of agent we evaluate its performance with the conservative data sharing, direct data sharing and no data sharing.

3 Experiment

3.1 Basic settings

We conduct all the experiments based on the MuJoCo Walker2D environment with 2 types of main task (walk, run) and 4 types of main data set.

All models have been trained for a total of 50,000 steps. The evaluation is conducted every 1,000 steps.

The detailed setting of hyper-parameters shared for all models are shown in Table 1

Parameter	**Value**
Actor, Critic	MLPs(3 hidden layers, 1024 units)
Action activation	Tanh
Optimizer	Adam
Learning Rate(Critic)	3×10^{-4}
Learning Rate(Actor)	1×10^{-4}
Soft Update Rate	0.01
Discount Factor	0.99
CQL target penalty	5
Evaluation episode	10

Table 1: Configuration

3.2 Evaluation metrics

In the evaluation of our reinforcement learning models, we utilize two specific terms to quantify performance metrics:

- **Max Reward(MR):** This term refers to the maximum reward achieved by the model during the evaluation phase. It is a crucial metric as it indicates the peak performance of the model under the given evaluation conditions.
- **Step Index of Max Reward(S):** This represents the index of the step at which the model attains the maximum reward, denoted as $S(k)$. This metric provides insights into the model's efficiency of convergence.

The details of the maximum evaluation reward of different baseline models under different data share method are shown in Table **??**

3.3 Influence of Data Sharing

To investigate the effectiveness of our proposed Conservative Data Sharing (CDS) method, we conducted experiments comparing it against single-task learning and a baseline direct data sharing method. These experiments were carried out across multiple agents (TD3-BC, BCQ, CQL, BC-CQL) and dataset types.

Figure 2 illustrates the reward-step curves for each data sharing method based on CQL algorithm trained on an expert walk dataset.

	DS Method	Walk		Walk-r		Run		Run-r	
		MR	S(k)	MR	S(k)	MR	S(k)	MR	S(k)
CQL	no share	**985.15**	31.5	**969.67**	22	283.86	35	**292.87**	38.5
	direct	955.76	47	951.98	20	**304.85**	43	267.71	33.5
	conservative	969.52	47.5	945.54	45	254.63	37	259.74	40
BCQ	no share	26.42	42	31.20	39	26.17	40	29.48	10
	dire	31.82	41	31.82	41	30.70	24	30.70	24
	conservative	34.99	37	38.17	40	35.91	33	40.81	43
CQL-BCQ	no share	955.37	41	933.39	29	-	-	-	-
	direct	869.04	**30**	869.04	30	-	-	-	-
	conservative	843.86	**31**	870.12	47	-	-	-	-
TD3-BC	no share	850.26	38	678.65	42	237.88	48	268.89	45
	conservative	776.17	41	726.90	43	188.96	50	248.37	38

Table 2: The reward of different algorithms using different data sharing strategy. "MR" means the max reward during evaluation. "S(k)" means the index of step at which the model takes the max reward."no share" means the single task, "dire" means directly share all data in multi-tasks for agent. "conservative" means employing the CDS data sharing method. "Walk-r" and "Run-r" represents the expert data, "Walk", "Run" represent the normal data.

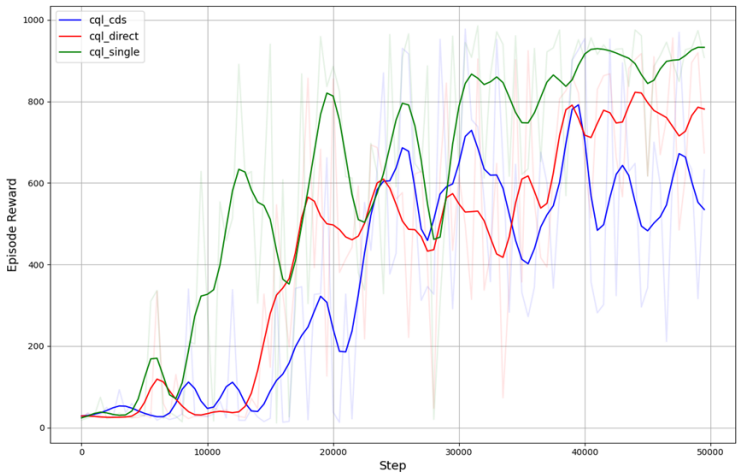


Figure 2: CQL based reward-step curves for different data sharing methods (walk)

- **Single Task Performance (Green Line):** This curve represents the performance of the CQL agent when trained exclusively on the walk dataset, serving as the target benchmark for multi-task learning scenarios.

- **Direct Data Sharing (Blue Line):** This curve shows the initial performance of the CQL agent using a multi-task dataset without any specialized data sharing techniques. It serves as the baseline for our experiments.

- **Conservative Data Sharing (CDS) (Red Line):** Represented by the red curve, this method involves the use of a selective replay buffer as proposed in our CDS strategy. Notably, the agent's evaluation reward ascends more rapidly at the beginning of the training process compared to the direct method and maintains a higher performance level towards the end of the training. However, it still does not surpass the single-task performance.

3.4 The Impact of BCQ on CQL

The following analysis focuses on the evaluation of the BC-CQL (green line) algorithm's performance in comparison to the standard CQL (blue line), BCQ and TD3-BC algorithm.

Figure 3 illustrates the reward-step curves from our experiments with CDS data sharing method on walking task.

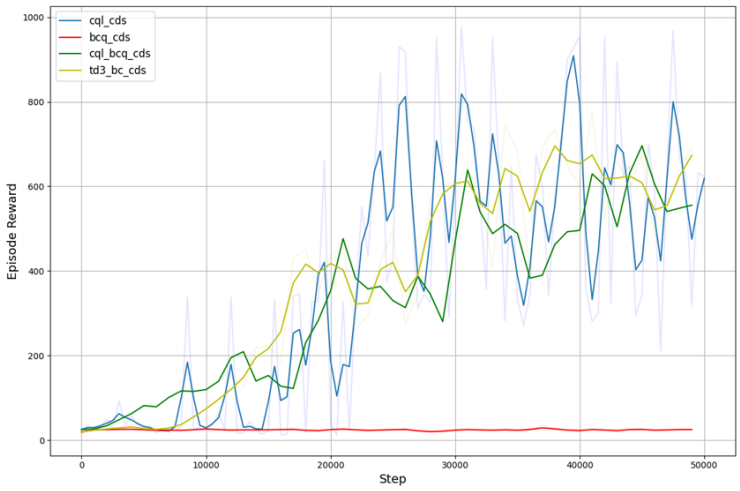


Figure 3: Reward curves for different baseline algorithms based on the CDS method (walk)

Due to certain issues with implementation details, We were unable to obtain an effective BCQ model.

Represented by the blue line, the CQL algorithm shows higher variability in its reward-step curve. This variability suggests a greater susceptibility to the common pitfalls of offline reinforcement learning, such as overestimation of the action values outside the support of the data distribution.

The performance curve for the CQL-BCQ algorithm is represented by the yellow line. It demonstrates more stable behavior throughout the training steps compared to the standard CQL algorithm. This stability is indicative of the BCQ component effectively mitigating the extrapolation errors that are typically present in offline reinforcement learning setups.

3.5 Samples

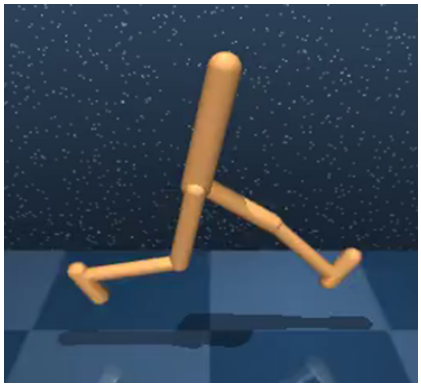
We conducted a detailed analysis of the performance of the BC-CQL algorithm compared to other agents by examining both high-performing and atypical samples from our offline dataset and evaluation process. Figure 4 shows some examples.

Upon visualizing the offline dataset, we observed distinct behavioral patterns for different tasks. For the walking task, it was noted that the agent primarily relied on one leg for movement, effectively rendering it as functioning with a limp. Conversely, in the running task, agents failed to learn effective running strategies. Instead, they developed a technique of propelling themselves forward by pushing off the ground with their knees. See Figure 4(d)

Initial stages of training revealed that the agents displayed a limited range of actions, often resulting in extreme behaviors such as falling headfirst or remaining stationary. (See Figure 4(b))This lack of flexibility suggested an initial inability to adapt to varied environmental contexts or challenges.

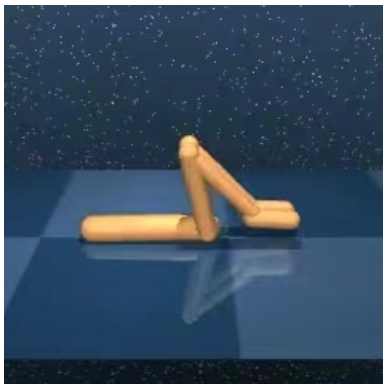
Our analysis suggests that a crucial component for successfully completing walking or running tasks is the agent's ability to recover and stand up from diverse postures and positions. While the expert dataset provided guidance on recovery maneuvers, typical walking or running behaviors remained largely unmastered by the agents.

As the training progressed and the rewards converged, both the walking and running agents began to exhibit similar behaviors, predominantly walking with one active leg. Initially, it was hypothesized that the knee-propulsion behavior was a precursor to walking. However, upon further reflection, we



((a)) Good example

((b))



((c)) Walking Task Final Stage

((d)) Running Task Final Stage

Figure 4: Visualization

concluded that this was indicative of over-fitting within the running task dataset. Once the agents mastered the skill of standing up, they commenced walking; lacking this ability, they continued to misinterpret kneeing as a running action.

4 Conclusion

We introduced a novel approach named Batch Constrained Conservative Q-Learning (BC-CQL) aimed at addressing the challenges associated with multi-task offline learning. This method enhances the stability of the learning process by significantly reducing extrapolation errors inherent in offline reinforcement learning models. Our comparative analysis with three classical offline learning algorithms—TD3-BC, BCQ, and CQL—demonstrates that BC-CQL not only achieves faster convergence but also maintains greater stability across various tasks.

Furthermore, we explored the effectiveness of CDS data sharing method within the context of these algorithms. By optimizing data utilization across a multi-task dataset, BC-CQL was able to improve the quality of the learned policies, reflecting a more efficient use of available data.

However, it is important to note that the training duration for our experiments was limited, which may have constrained our ability to fully evaluate the long-term efficacy of the proposed methods. Future work will focus on extending the training periods and conducting more comprehensive evaluations to better assess the potential and limitations of BC-CQL in more complex and varied task environments.

5 Acknowledgement

All participants contributed equally to this project.

Xiaohan Yu developed the data sharing method using CQL and the TD3-BA agent and authored the introduction section of the report. Yuchen Hou implemented the BQL agent and applied the data sharing method to it, completing the experimental section of the report. Siyu Pan designed the BC-CQL agent and incorporated the data sharing method into it, crafting the algorithm section of the report.

References

Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without exploration. In *Proceedings of the 36th International Conference on Machine Learning*, 2019. doi: 10.48550/arXiv.1812.02900.

Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative q-learning for offline reinforcement learning, 2020. Submitted on 8 June 2020, last revised 19 August 2020.

Tianhe Yu, Aviral Kumar, Yevgen Chebotar, Karol Hausman, Sergey Levine, and Chelsea Finn. Conservative data sharing for multi-task offline reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 34, pages 11501–11516, 2021.